



Module 30

The Verification Completeness Problem

cādence[®]

It's important to understand what the term verification completeness means and why it's such a big problem. And in many ways, it's something that can never be achieved. It's important for any verification course that we cover these kinds of concepts.

The example we're going to follow through is about formal verification, a very simple example just to show you how hard this is to achieve completeness.

Objectives

In this module, you will:

- Review verification methodologies.
- Examine a case study that highlights the verification completeness problem.
- State techniques for minimizing the risk of verification holes.
- State the merits of each verification technique.



We will start off with a review of verification methodologies, and the case study will be specific to formal. I minimize the risk of missing a bug during verification.

Scale of the Functional Verification Problem

The most complex processor IC designs may have, for example, 500 thousand flips-flops (FFs). Those FFs could be in any of 2^{500000} different states at any particular time.

The actual number of states will be much less because:

- By design, not all states will be possible.
- There will be many states we don't particularly care about.

However, the number of states is still absolutely massive.

It is essential that you accept this truth.

You cannot possibly test everything!

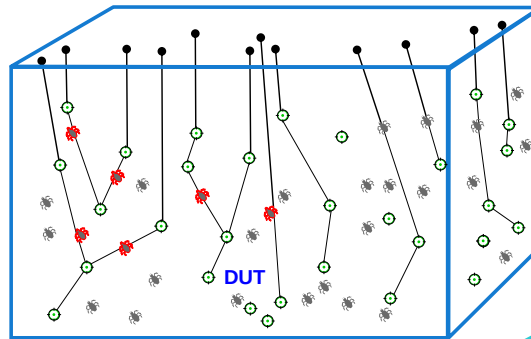


The scale of the problem is really at the root cause of why it's so hard to be sure you've covered everything. For the most complex ICs of today, you might have something like, for example, 500,000 flip-flops, which is a lot. And if you have 500,000 flip-flops, it means you have a possibility of two to the power 500,000 different states at any particular time. Now, of course, that theoretical maximum will never be achieved because, by design, most states will not be possible, or many of those states rather. And there are also many states we don't actually care about because they don't represent real-life operations. Even so, we've still got a huge state space to explore. The first thing to accept is that you can't possibly test everything, and if you're a simulation user, whether you like it or not, that's the decision you already made. If you're using simulation you've recognized, you can't possibly test everything. The problem then becomes one of minimizing the risk of bugs and escaping verification.

Most Basic Approach to Verification: Directed Tests

Verification Engineer:

- Defines DUT states to be tested based on specified behavior and corner cases.
- Writes directed test to visit and verify each item in the test plan.



~~Automation~~

Significant manual effort to write all the tests.

~~Automation~~

Work required to verify that each goal was reached.

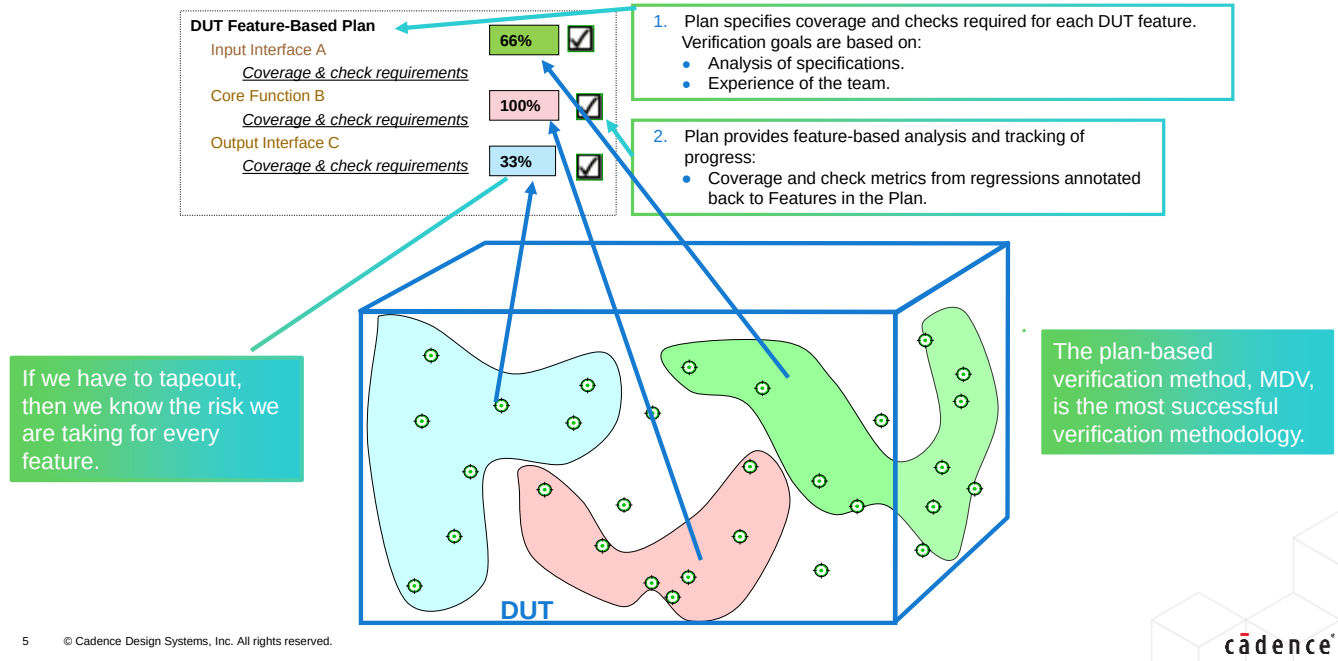
~~Completeness~~

Poor coverage of non-goal scenarios ... especially the cases that you didn't "think of."

Redo if design changes

A history lesson on verification. It started off with specific director tests, so you had to sit there and think of what stimulus to apply and apply that to the design and observe the outputs and check it's behaving correctly. This was a very labor-intensive task because you have to think of and write all tests, and if anything changes, like the design changes, you have to do it all again. You had very poor coverage of non-goal scenarios. It's really the things you don't think of that catch you out, not the things you did think of.

Metric Driven Verification (MDV)



This kind of evolved into a verification methodology driven overall by coverage, and that coverage is defined in some hierarchical verification plan.

So, this is what this is showing here, a DUT feature-based plan. For all the features that are required of the design, we come up with a list of all the different kinds of checks on the different kinds of coverage we need in order to be sure that we verify the design functionality is as expected. These plans are derived based upon the experience of the team, and analysis of the functional spec and what this ends up being is that if you can imagine your whole design now,

this is the entire state space represented here for any given feature; this will cover a certain part of the design. Another feature will cover another part of the design, and another feature will cover another part of the design in terms of both checks and coverage. And we're hoping that adding all these up means we've tested the entire design. Essentially, that's what we're doing. And the advantage of doing it in a hierarchy-based plan like this is that you've got an up-to-date measure, i.e., as you're doing each test, you can update this to see how much of each feature has been tested and so forth. The whole idea of any kind of verification, whether formal or simulation, is that you want to achieve the maximum amount of coverage you can 100%, ideally in the context of no checks failing.

We can view these metrics here as being an indication of how much risk we take. If we tapeout today, verification is about risk management because we're already accepted. We can't test everything.

Concise and Simplistic Summary of MDV

Define a hierarchical verification plan, which:

- States the features the design is supposed to have.
- Specify the checks to be done in order to test those features.
- Specify the coverage required in order to be confident that we tested every important scenario the design is expected to work under.

We know we have finished verification when we have reached 100% coverage in the context of no checks failing.



In order to state what MDV is for the simulation world, this is the best methodology anybody has come up with so far. We devise a hierarchical verification plan which states the features the design is supposed to have, which we get from the functional spec. We specify the checks which need to be done in order to test those features and the coverage required in order to be confident. We tested every important scenario that the design is expected to work under, and our hope is that we reach 100% coverage in the context of no checks failing. Now we might have to settle for a number lower than 100 due to project deadlines like tapeout.

“The Best Laid Plans of Mice and Men”

As the saying goes, “If we fail to plan, then we plan to fail.”

However, having a plan does not guarantee success.

The plan could be wrong.

The plan might not be implemented correctly.

To show how this can happen for even the simplest of designs, we will follow a case study.

The schematic we will show defines the functional specification.

The intent is to verify every feature of the design by defining SVA properties.

We will verify these properties using JasperGold Formal Verification.



However, just because we've got a plan doesn't mean everything is going to be okay because the plan might be wrong. The plan is susceptible to the same human errors that everything else is. The plan might be correct but not implemented correctly, so everything could still go wrong. What we're doing, in this case study is showing you an example of such a case. What we want to do is show a schematic that defines the functionality required. So effectively, the schematic represents what the functional spec is, and the intent is that we verify the features that the design has, and in this case, we will use JasperGold because this is a formal course.

Completeness of Property Sets

We obviously define what we think are sufficient properties:

- As identified in the verification plan.

This is not just limited to properties or formal verification.

- The same argument for a constrained random coverage-driven simulation testbench:
 - Did we specify enough coverage?

How can we be certain they cover all required functionality?

A formal tool might tell us that assertions cover all I/Os and FFs.

This does not mean that we have written properties to cover all functionality.

- In the same sense, that code coverage does not tell us we have complete functional coverage.



If we're using JasperGold, we're going to write some properties in order to check our behaviors. Correct. And those checks are being derived from the verification plan.

The reasoning we will apply here doesn't necessarily only apply to formal. You could make similar arguments for simulation, like if I've got a constrained random stimulus testbench, which is coverage driven, our goal is to fill coverage. How do we know that the coverage is correct? You can point to anything and say that might be wrong.

So how can we be certain they cover all the required functionality? Now you do have some help from the tool. It's not like the tool tells you anything. A formal tool, we could get reported, for example, that every IO and every flip-flop is covered in the funding cone of some assertion and was used effectively using something called proof core in JasperGold.

This does not mean with enough properties, however, to cover all functionality, even if the metrics we get from this tool tell us, yes, everything has been covered just in the same sense that if we got code coverage, it doesn't tell us we've got complete functional verification.

Verification [in]Completeness Example

Shown is a schematic of the functional requirements that are implemented in RTL:

- We write blackbox properties, which we think completely check all desired functionality.
- Human error or oversight might mean that we don't specify all desired behavior.

Tool might tell us:

- All I/Os and all FFs are included in some assertion:
 - There is no deadcode
 - All checks pass
 - All cover items pass

Should we conclude that verification is complete?

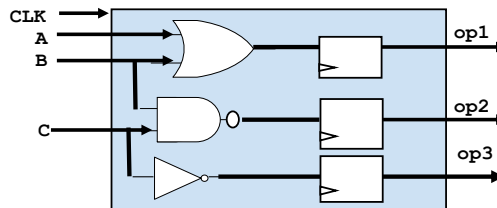
- No way of knowing that we specified enough properties to check all functionality.
- Human error or oversight might escape detection in the verification process.

At first glance these properties seem sensible, however

```
property P1 is always ( (A || B) ==> op1 ); ✓
property P2 is always ( (!A && !B) ==> op2 );
property P3 is always ( !C ==> op3 );
```

Human error. A has no effect upon op2 but property still holds.

a	b	c	op1'	op2'	op3'
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	1	0	0



9 © Cadence Design Systems, Inc. All rights reserved.

Here's an example of the schematic we've got here. Those are the requirements, and this is a property set we hope will check all behaviors of the design at the first inspection. These look like a perfectly reasonable set of properties, right?

For example, in this property, I say if I've got A or B on the next cycle op1 should be true. That would appear to check this all-gate and this flip-flop here. And likewise for the other outputs.

However, there might be human error or oversight, which means we didn't capture that behavior correctly. The intended check is not specified correctly so in this example. For P-2, it says Not A or not B implies OP2 is true in the next cycle. Now notice op2, A does not appear in the funding code for that. A has no influence over what OP2 is doing; however, It just so happens, and we can check this by looking at the truth table if we wish; that property will always hold, i.e., always be true.

So even though the tool might tell us that all I owe is a flip flop included in some assertion and that there are no branches of code that can't be reached, and all checks pass. All cover items pass, i.e., 100% coverage in the context of no checks failing, which I said was the end of verification. We still haven't verified our design correctly due to human error, which the tool brought to us. The question is that what do we do about that then?

Design Mutation

Shown is a schematic whose behavior satisfies our properties.

This is a mutant of the desired design.

We can see that our verification environment fails to detect this mutation of our desired design.

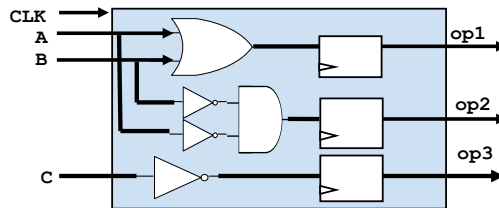
Hence, we did not capture a property set that completely and uniquely defines desired behavior.

- However, maybe we never intended to allow flexibility for the design implementation.

Change from
Desired Truth
Table

a	b	c	op1	op2	op3
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	0	0
1	1	0	1	0	1
1	1	1	1	0	0

```
property P1 is always ( (A || B) ==> op1 ); ✓
property P2 is always ( (!A && !B) ==> op2 );
property P3 is always ( !C ==> op3 );
```



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

In this example, we can (if the properties are sufficient) detect the mutant design.

In practice, the mutation might not propagate an observable difference from the expected behavior to somewhere it could be detected.

This indicates a problem with complete stimulus; that is, we did not provide the correct stimulus to observe the difference in behavior caused by the mutation.

Now, if we imagine that we make a change to the design, we might expect that these properties, if they completely define the required behavior, would fail. So that's called design mutation.

What we've got here now, we've changed that schematic. Now, if I just go back to the previous slide for a second. Pay attention to the schematic diagram. It's changed from that to this.

Now it just so happens that the change we've made will not be detected by any of these assertions, i.e., if all the previous assertions passed there are going to be no failures here. I've managed to change the design without my properties detecting the design has been changed.

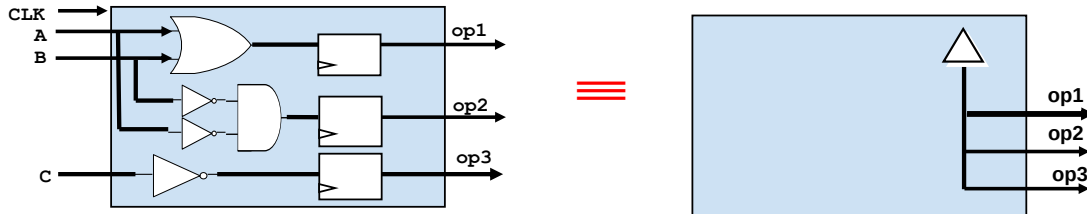
This comes down to what is really a philosophical question do we really want a property set that you completely and uniquely define the desired behavior? It might be the case that we don't care if the design can be changed and none of the properties fail because we've captured all of the required behavior that's important to us inside of these properties. But again, that requires the tool understanding what our intent is, which obviously it can never do, a tool is never going to be able to read our minds.

So again, we could follow that through in a truth table. You notice these bits which are kind of grayed out, these zeros here, there, there, and there. Those are changes in the truth table from the previous one. And it just so happens that all of these properties still hold even with that design mutation.

We Don't Even Have the Fundamentals Correct

All these properties pass with the outputs tied to the power rail.

```
property P1 is always ( (A || B) ==> op1 ); ✓  
property P2 is always ( (!A && !B) ==> op2 ); ✓  
property P3 is always ( !C ==> op3 ); ✓
```



The correct assertion to check OP1 is not particularly intuitive.

```
property P1 is always ( op1 == $past(A || B) ); ✓
```

And when you think about it, it's even more; it's even deeper than that. It's more fundamental than that because all of these properties will pass if all outputs are tied to the power rail instead of this required circuit. Somebody could present us with this so all the properties would still pass. They can never fail, can they? Because these outputs, one, two, and three, will always be true.

In order to correct this property P1, for example, what we would have to do is this if we wanted to do it in single assertion and this solution is not exactly intuitive, most people would not be able to come up with this by themselves without having seen a similar example. What we're saying here is OP1 must always equal the past value of this expression or be past one cycle ago, and being as that's a flip-flop, this satisfies that requirement now. But again, it probably wouldn't be our first guess. It's very unlikely we'd come up with this unless we'd seen a similar property before.

Locality and the Dangers of Over Optimism

If one made JG report elements in the COI of the assertion, then it would be both the grey and green parts.

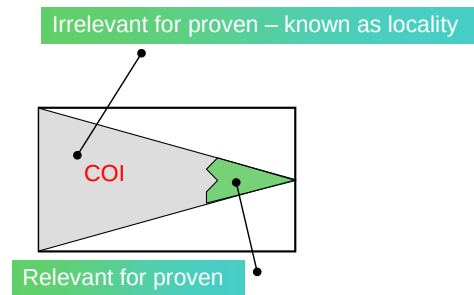
- For example, the static fanin cone of all signals in the assertion.

Question: Does this mean we have covered/verified/exercised all elements in the COI of the property?

There is the danger of over-optimism on coverage.

The grey part is irrelevant for the proof.

We learned nothing about the grey part.



12 © Cadence Design Systems, Inc. All rights reserved.



Knowing how much of the design was meaningfully exercised by JasperGold is a very important metric for just about every customer.

The coverage app can collect design metrics on:

- Stimuli coverage
- COI coverage
- Proof coverage
- Bound coverage
- Code Coverage
 - Branch
 - Statement
- Functional Coverage
 - SVA/PSL covers

Contact your local AE to discuss the JasperGold Coverage app.

Now, with the report on, you know, for formal, we've got a code of influence here. And it might be that the entire code of influence, which is in effect, the structural funding code for the signals and the assertion, it's not required in order to prove the assertion. It's irrelevant as regards the proof. Obviously, these signals can influence what these signals in the assert do, but they can't change the result of the assertion, which is a completely different thing. It could be the case that this gray portion of this code is not actually required for the proof. It doesn't matter what it is in effect. We've not tested it.

Okay. So how do we know that? We're not being overoptimistic?

If we just looked at the structural function code and says, yes, we tested it, the answer is, well, no, it might not be needed. What we do in Jasper is use the just for gold coverage up in order to tell us this, which is what we said on the previous slide just for gold coverage up.

Can We Achieve Completeness?

Is it impossible for a tool to read our minds?

Probably, but we can get closer using tool features.

No current tools can tell you when you have a complete property set.

There is no substitute for:

- Understanding the requirements
- Intelligent planning
- Attention to detail
- Hard work

No EDA tool can tell you that your verification is complete.

All the EDA tools can tell us is if we didn't test all the things that we said we would, namely if we are incomplete.



The question being asked is how can we achieve completeness, i.e., can the tool tell us that we've got complete verification? Well, the answer is no, because the tool can't read our minds. And effectively, what you're describing is the same problem as just the tool creating the design for us. If there was a completely unambiguous way of specifying the requirements, and that's not going to happen in the near future, of course, but we can get closer to that minimize risk using the different tool features we have.

So regardless of what anybody claims, no tools can tell you. You've got a complete property set.

There's no substitute for doing all the things only humans can do, which are listed right there. However, what anybody can tell us is we didn't test all of the things that we said we would, which at least gets us part of the way toward understanding whether we're complete or not.

Assessing Completeness

Defining your verification objectives upfront is the best way to assess when you are done.

Beyond this, there are other ways to assess incompleteness:

- Do all the outputs of the block have assertions describing their behavior?
- The JasperGold Coverage app can help assess what portion of the design your assertions are exercising:
 - Cannot tell you when you are done, but when you are *not* done.

There are no tools that can tell you when you have done enough verification at present.



So, in assessing completeness, you really have to measure this against what you've defined as being a complete set of tests. Now, what the tool can tell you is whether you're incomplete in your verification, like do all the outputs of a block have assertions describing their behavior? JasperGold coverage can tell us what parts of the design were and were not exercised during all proofs and all covers. So essentially, what they are telling you is that you've not finished. Not that you are. Incompleteness is an easy thing, knowing you are incomplete; completeness seems to be an intractable problem with the present technologies.

Verification Execution Completeness

This is something that we can know.

The goal is the same in both dynamic (simulation) and formal.

All check items pass/proved in context of coverage items *equals* verification complete.

Execution Method	Execution Item	Completeness	Execution Method	Execution Item	Completeness
Dynamic	Check	All checks are executed with no violation (checks pass)	Formal	Check	All checks are proven with no violation (checks proved)
	Coverage	All coverage items are reached (pass); driver + DUV dependent		Coverage	All coverage items are reachable; constraints + DUV dependent

15 © Cadence Design Systems, Inc. All rights reserved.



We can know for sure that all checks we have defined have passed and we have reached all cover items.

The holy grail of verification is knowing whether those checks and cover items defined all behaviors we want to verify.

This might be possible in the future but not at the present time.

But what you do know is how much you've executed.

So regardless of whether you're doing dynamic verification, which is simulation and emulation, your requirements in that are typically you define some checks and you just find some coverage, which is all the expected scenarios, the designs expected to operate under. And you wish to get 100% coverage in the context of all checks passing in formal. That's exactly the same.

Although in formal coverage means a slightly different thing, it uses exactly the same syntax as simulation. A coverage in formal tells you whether that scenario is ever possible. It's the same situation in formal. Your checks without your coverage are useless. You need 100% coverage in the context of no checks failing.

What Can Be Automated?

It's probably more important to ask the question in reverse, what can't be automated?

- Writing the functional specification
- Writing the verification plan
- Interpreting the verification plan to implement the checks and coverage
- Interpreting the results from verification runs
- Understanding the limitations of the verification technique we use

EDA is about automating all that can be, for example:

- Managing huge amounts of data
- Compilation/recompilation
- Running and rerunning verification tools after bug fixes
- Analyzing which tests are the most valuable in terms of coverage achieved
- Tracking bug rates/coverage accumulation rates



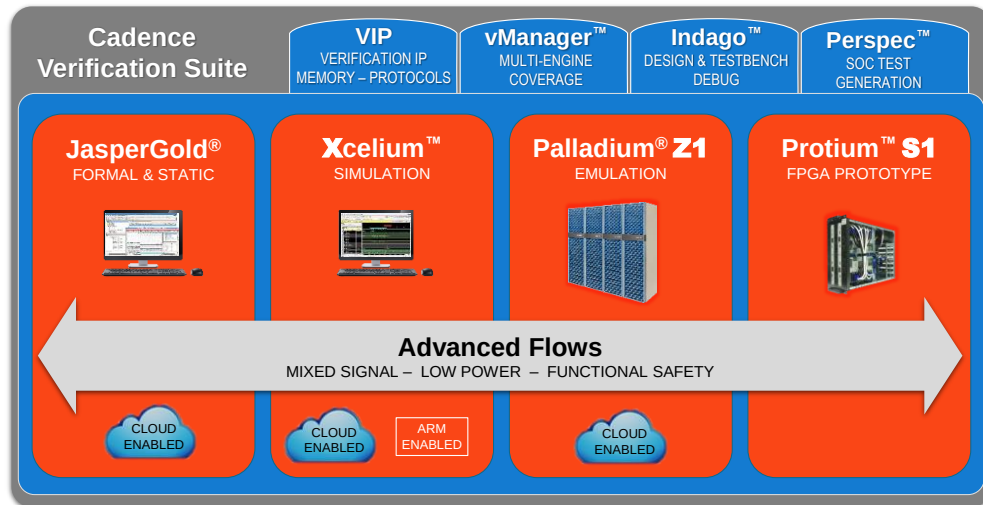
What can be automated, then? Despite all the claims you see in the media for machine learning and AI, the world isn't as advanced as it tells you on the news when you ask what can be automated. And a more important question, and a more realistic one, is what can't be automated? What can't be automated is writing the functional spec in the first place and the verification plan. Interpretation of that verification plan into the individual checks and coverage. It's often the context under which the results were received during the verification runs. And understanding the limitations of any technology we're using.

So clearly, that's why different technologies exist, like formal and simulation and hardware, acceleration, emulation, FPGA prototyping, and so on. There are all these different ways of doing things because not one methodology or technique does everything. So basically, there is no single solution.

Eda is about automating all that can be. We're talking about masses of data because you're talking about huge designs which create huge amounts of data. We might have to edit things and compile and recompile, which might take hours running verification runs, and if they fail, rerunning them after bugfixes, analyzing which tests give us the most information in the shortest time, tracking our bug rates, and our coverage accumulation. Those are the kinds of things that EDA does.

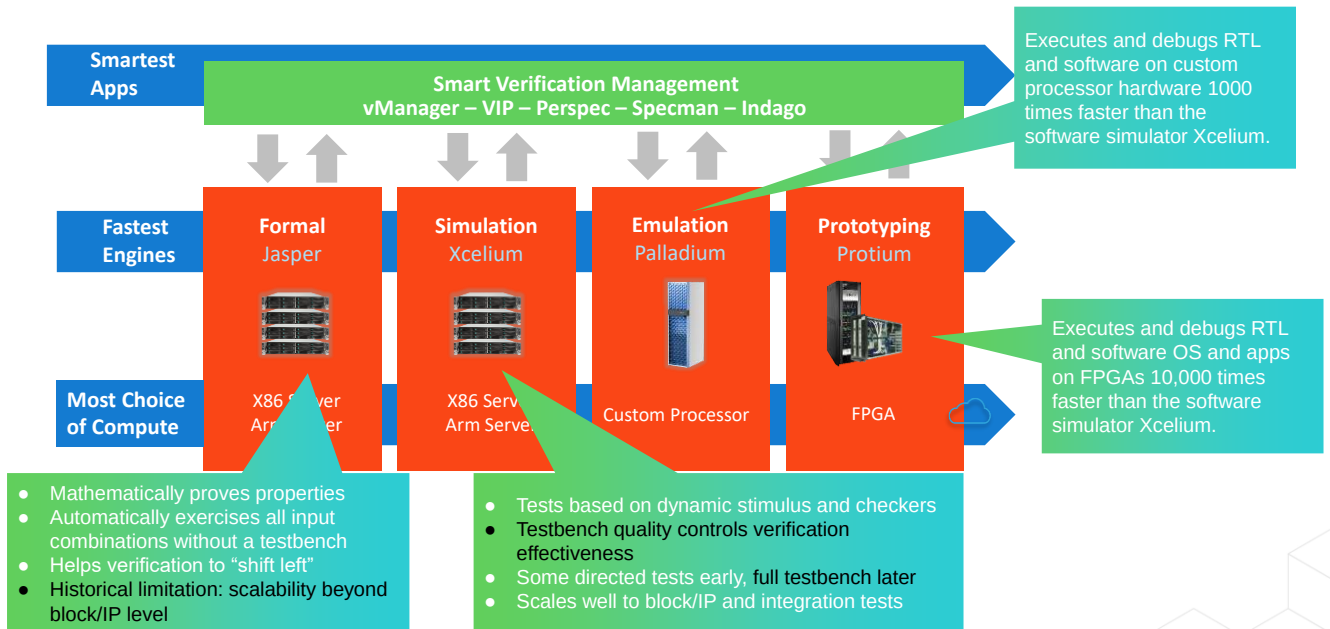
There's no magic tool that you can just press one button and say, Go and make my chip for me.

Cadence Verification Solution



This is the reason Cadence has a verification suite. Some techniques are appropriate at different levels of integration as well, so formal and static verification and mixed signal, low power, functional safety, all of these things all have to be done.

Where Each Technique Apply



And this is how they fit into where each technique applies.

So formal are mathematical and algebraic proofs. And it exercises all import combinations without the need for a testbench, and it helps us actually start verification while we're still designing. And a historical limitation is scalability beyond the block and IP level. If you're looking for exhaustive proof, simulation is the oldest form of verification. They started with the director tests, but really most people are using the metric-driven verification environment, which uses random stimulus on the inputs, and it scales well into block and IP, and integration tests. The problem becomes speed.

Emulation is a way of running a lot quicker, so thousands of times faster than a software simulator like Xcelium would.

Prototyping is when you're trying to debug the OS and software, which is tens of thousands of times faster than software and simulations. If you want to see your operating system boot, for example, you will be waiting an awfully long time if you do a simulation of this. In contrast, if your RTL is relatively stable, you can put this into an FPGA array effectively, which is like prototyping your entire chip, and you can run software in real-time on top of it.

Customer Needs: Emulation and Prototyping

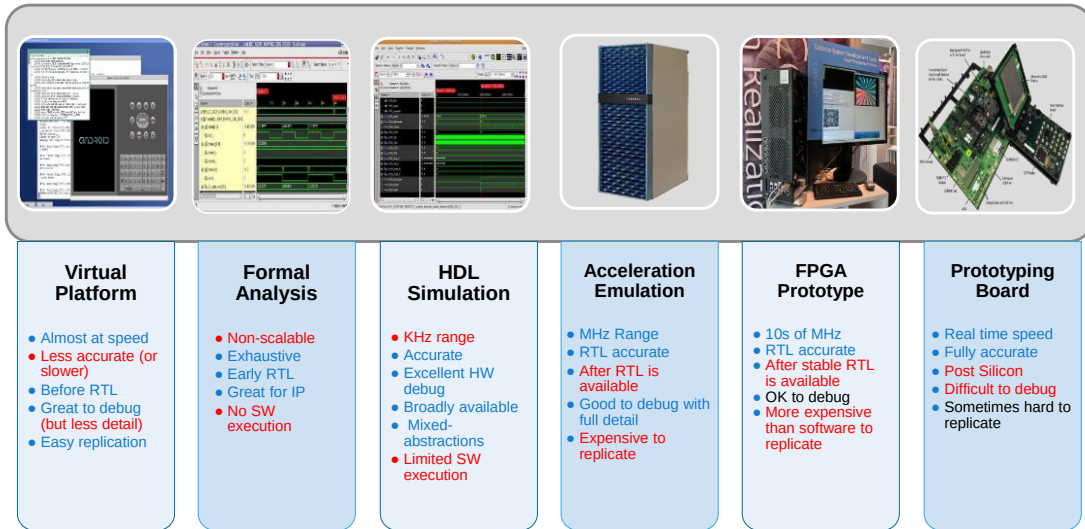
	Key Care Abouts
Emulation “Debug your design”	• Predictable fast build: Rebuild new RTL bug fixes every night. • SOC level capacity: IP level simulations typically run fast enough already. • Fast and complete debug: Full trace, runtime debug controls.
Prototyping “Debug your software”	• Build time and debug less important: Design stable, rebuild only every 1-2 weeks. • Highest performance: Software debug requires even longer simulation runs. • Lowest cost: Replicate one build for many different software developers. • Some SOC level capacity: More and more software complexity at the SOC level.



This page does not contain notes.

There Is No “One Size Fits All”

Verification and Software Platforms Need to Interoperate



20 © Cadence Design Systems, Inc. All rights reserved.

cadence

There is no magic silver bullet, no one size fits all. You need a combination of all these things in order to verify today's complex designs.

Here's showing you a kind of going from left to right probably in terms of time in the project. So here we're doing kind of simulation software simulations using less accurate models.

These models here, virtual platforms, and so on, we can exercise that. Our software works correctly in our OS, but we don't have the information on the implementation of the chip is lower, low enough level of detail that the tools can actually produce a kind of blueprint for the chip.

This is before writing RTL code. The RTL code is what we have to write to give us enough detail to actually manufacture a chip in real life. These two things on the left are all at a very high-level software level.

We then start designing the blocks from the bottom up. Using formal simulation and then HDL simulation, you're talking about as if you've got a clock frequency of kilohertz. It takes an awfully long time to simulate even a second's worth of real-time.

At that point here, the acceleration emulation helps because we can get up to say one megahertz kind of range as if we got a one-megahertz clock. So typically, you know, the final chip is probably going to have a clock of around two gigahertz or something similar to that. So, we're still orders of magnitude off the real-life speed here. This is cycle accurate. We can debug this at the level of detail we need.

And FPGA prototyping gets to the tens or hundreds of megahertz when we can actually run our design, which we think is pretty stable at this point in order to verify our software. Websites and apps that go on top of that, of course.

Summary

In this module, you learned:

- Functional verification is about risk management, not verifying everything.
- We need to minimize the chances of a critical bug escaping the verification process.
- There are many tools and methodologies to help us do this.
- We cannot abdicate the responsibility for thorough verification to automation.
- Automation only helps with managing data, predictable, repetitive tasks, and computing.
- Functional verification is still a human process vulnerable to human error.
- There is no single verification methodology which works best for every circumstance all of the time.



In summary, it's important to understand this from the very beginning. Functional verification is not about completeness. It's about risk management. You can't possibly test everything.

So, we just concentrate on what features are important to us and minimize the chances of bugs escaping.

And we can do that by using lots of different tools and methodologies.

Automation doesn't abdicate our responsibility to make sure everything is correct.

There is no solution to replacing humans, so automation only helps to manage the huge amounts of data and analyze the huge amounts of data that these verification tools will create.

There is no single verification tool you buy that is best for every circumstance. Each has its own kind of advantages and disadvantages. That completes this section. There is no lab exercise associated with this, so please move on to the next section.